

Student Record Management System

1. Project Overview

The Student Record Management System is a menu-driven application developed in C programming to manage student records using a singly linked list.

The project demonstrates the use of dynamic memory allocation, linked-list operations, searching, sorting, file handling, and modular programming.

The application allows the user to add, delete, display, modify, save, sort, delete all, and reverse student records.

The final executable should be named:

student

Student data is stored in a file named:

student.dat

2. Objectives

The main objectives of this project are:

- To implement a student record management system using a singly linked list.
- To understand dynamic memory allocation using malloc() and free().
- To perform insertion and deletion operations on linked-list nodes.
- To search student records using different criteria.
- To sort student records by name and percentage.
- To save and restore student records using file handling.
- To implement the project using separate source modules.
- To properly release dynamically allocated memory before program termination.

3. Student Record Structure

Each student record is represented using the following structure:

typedef struct st

```
{  
    int rollno;  
    char name[20];  
    float percent;  
    struct st *next;  
} SLL;
```

4. Functions, Working and Results

4.1. Add Function :

Function Prototype

```
void add(SLL **);
```

Working

The Add function creates a new student node using dynamic memory allocation. It finds the smallest unused roll number, accepts the student's name and percentage, and inserts the new node at the end of the linked list.

Sample Output

Enter your choice: a

Enter name & Percentage: Rahul 85.5

The student is added successfully with an automatically assigned roll number.

4.2. Display Function

Function Prototype :

```
void display(SLL *);
```

Working

The Display function traverses the linked list from the first node to the last node and displays the roll number, name, and percentage of every student.

Sample Output

```
-----  
Student Records present are:
```

```
-----  
1 Rahul 85.500000  
2 Anjali 92.500000  
3 Kiran 78.000000
```

If there are no records:

No records found

4.3. Delete Function

Function Prototype

```
void del(SLL **);
```

Working

The Delete function allows the user to delete a student either by roll number or by name. When deleting by name, matching records are displayed and the required record is selected using its roll number. After deletion, the memory occupied by the node is released using free().

Sample Output – Delete by Roll Number

R/r : Delete by roll number

N/n : Delete by name

Enter your choice: r

Enter roll number: 2

Record deleted successfully

Sample Output – Delete by Name

R/r : Delete by roll number

N/n : Delete by name

Enter your choice: n

Enter name: Rahul

Roll No: 1 Name: Rahul Percentage: 85.500000

Enter roll number to delete: 1

Record deleted successfully

If the record does not exist:

Record not found

4.4. Modify Function

Function Prototype

```
void modify(SLL **);
```

Working

The Modify function searches for a student using roll number, name, or percentage. If multiple records have the same name or percentage, the matching records are displayed and the user selects the required record using its roll number.

The name and percentage are then updated.

Sample Output

R/r : Search by roll number

N/n : Search by name

P/p : Search by percentage

Enter your choice: r

Enter roll number: 1

Current: 1 Rahul 85.500000

Enter new name & Percentage: Rohan 90.5

Record modified successfully

After modification:

1 Rohan 90.500000

4.5. Save File Function

Function Prototype

```
void save_file(SLL *);
```

Working

The Save File function opens student.dat and writes the details of every student into the file. The linked list is traversed and each record is stored in the file.

Sample Output

Enter your choice: v

File saved Successfully

The student.dat file contains data similar to:

1 Rohan 90.500000

2 Anjali 92.500000

3 Kiran 78.000000

4.6. Sort Function

Function Prototype

```
void sort(SLL **);
```

Working

The Sort function provides two options:

- Sort by name in alphabetical order.
- Sort by percentage in descending order.

Sample Output – Sort by Name

Before sorting:

1 Rohan 90.500000

2 Anjali 92.500000

3 Kiran 78.000000

N/n : Sort by name

P/p : Sort by percentage

Enter your choice: n

Records sorted by name

After sorting:

2 Anjali 92.500000

3 Kiran 78.000000

1 Rohan 90.500000

Sample Output – Sort by Percentage

Enter your choice: p

Records sorted by percentage

After sorting:

2 Anjali 92.500000

1 Rohan 90.500000

3 Kiran 78.000000

4.7. Delete All Function

Function Prototype

```
void del_all(SLL **);
```

Working

The Delete All function traverses the complete linked list and frees every node using free(). After deleting all nodes, the head pointer is set to 0.

Sample Output

Enter your choice: 1

All records deleted successfully

If Display is selected afterwards:

No records found

4.8. Reverse Function

Function Prototype

```
void rev(SLL **);
```

Working

The Reverse function changes the next pointers of the nodes to reverse the order of the linked list. No new nodes are created.

Sample Output

Before reversing:

1 Rohan 90.500000

2 Anjali 92.500000

3 Kiran 78.000000

Enter your choice: r

List reversed successfully

After reversing:

3 Kiran 78.000000

2 Anjali 92.500000

1 Rohan 90.500000

5.Overall Result

The Student Record Management System successfully performs the required student record operations using a singly linked list. The program supports adding, displaying, deleting, modifying, sorting, saving, deleting all records, and reversing the linked list. Dynamic memory allocation and deallocation are also used for proper memory management.